

1 Concept

Problem: Citizens frequently encounter urban issues like potholes, garbage accumulation, broken streetlights, and illegal parking. Existing complaint handling systems are slow, manual, and lack real-time monitoring.

Solution: An AI-powered Smart City Issue Reporting System that: - **User App:** Allows citizens to report issues using photos, descriptions, and GPS coordinates. - **Machine Learning:** Automatically detects issue type, severity, and generates bounding boxes for visual confirmation. - **Admin Dashboard:** Enables city authorities to track issues, assign tasks, and view analytics in real-time.

Impact: Faster reporting, automated categorization, reduced manual effort, and data-driven urban management.

2 Skills and Their Usage

Skill/Technology	Purpose / Use
React	Build user interface for both citizens and admin dashboard (forms, maps, status views).
Tailwind CSS	Stylish, responsive, and modern UI for forms, dashboards, and map overlays.
Formik + Yup	Handle form inputs and validations for issue reporting.
Node.js + AdonisJS	Backend API development, authentication, database interaction, business logic.
Redis + BullMQ	Queue jobs asynchronously for ML processing, ensures scalable issue processing.
PostgreSQL + PostGIS	Store issue metadata, user data, and geospatial coordinates for mapping and analysis.
AWS S3 / Cloudinary	Store uploaded images efficiently and serve them for frontend and ML inference.
Python	Core language for AI/ML model training and inference.
PyTorch / TensorFlow	Train object detection and classification models.
YOLOv5 / YOLOv8	Detect object type (pothole, garbage, streetlight) and bounding boxes.
MobileNetV2	Lightweight classification for faster, real-time inference (optional).

Skill/Technology	Purpose / Use
FastAPI	Serve ML models as API endpoints for backend requests.
Socket.io	Real-time notifications to users and admin dashboard.
Mapbox GL JS / Leaflet	Visualize issues on interactive maps with clustering and heatmaps.
Docker	Containerize services for easy deployment and development consistency.

3 Requirements

Hardware: Laptop/PC, Camera/Smartphone

Software: Node.js, Python 3.9+, PostgreSQL, Redis, Docker

Data: 200–500 labeled images per issue category, labeled in YOLO format

Users: Citizens reporting issues, City Admins monitoring and managing issues

4 Project Structure

```

/smart-city-project
|— /frontend      # React + Tailwind UI, forms, map, status pages
|— /backend      # Node.js + AdonisJS APIs, database models, job queue
integration
|— /ml_service   # Python FastAPI service hosting ML models
|— /worker       # BullMQ worker for asynchronous ML inference
|— /data         # Labeled dataset for training ML models
|— docker-compose.yml # Service orchestration

```

5 System Flow

1. **Report Submission:** User submits photo, description, and GPS location.
2. **Frontend API Call:** POST `/api/report` to backend.
3. **Backend Processing:** Save image (S3/Cloudinary), metadata (PostgreSQL), mark status `pending`.
4. **Job Queue:** Push report ID to Redis queue (BullMQ).
5. **Worker Processing:** Pull job, send image to ML service (FastAPI), ML predicts category, confidence, and bounding box.

- 6. **Database Update:** Update report entry with ML predictions, status `reviewed`.
 - 7. **Realtime Update:** Socket.io updates dashboard and notifies user.
 - 8. **Feedback Loop:** User feedback can be collected for retraining the ML model.
-

6 Detailed Features

User Features: - Submit issue reports with photo, description, GPS location - Track report status in real-time
- Receive push notifications on status updates - View city map with reported issues

Admin Dashboard Features: - Interactive map with clustering and heatmap overlays - View detailed issue information including ML bounding boxes and confidence scores - Review, approve, and assign issues to departments - Analytics: number of issues, types, locations, resolution times - Manual ML correction if predictions are inaccurate - Export reports in CSV/PDF

ML Features: - Automatic issue type classification - Confidence scoring for predictions - Bounding box visualization on images - Feedback integration for continuous model improvement

7 Development Plan

Week 1: - Initialize repository, frontend forms, map placeholders - Backend: PostgreSQL setup, upload API, Redis queue integration

Week 2: - Label images for ML training - Train YOLOv5/MobileNet models - Build FastAPI ML service - Worker integration for asynchronous ML inference

Week 3: - Admin dashboard: map, heatmap, cluster markers - Detailed report view with ML predictions - Real-time updates via Socket.io - Auto-assignment of issues based on category

Week 4: - UI polish using Tailwind CSS - Analytics dashboards with charts and trends - Test latency and accuracy - Prepare pre-recorded demo as backup - Deploy via Docker, Render, or Railway

8 Machine Learning Model & Training

Models: YOLOv5/YOLOv8 for object detection (bbox + category), MobileNetV2 for lightweight classification.

Training Steps: 1. Organize dataset in YOLO format 2. Apply transfer learning (30–50 epochs) 3. Evaluate precision, recall, mAP 4. Export trained weights 5. Deploy ML inference via FastAPI

9 Tech Stack Overview

Layer	Technology
Frontend	React + Tailwind + Formik + Yup
Backend	Node.js + AdonisJS + Redis + BullMQ
Database	PostgreSQL + PostGIS
Storage	AWS S3 / Cloudinary
ML Service	Python + PyTorch/TensorFlow + FastAPI
Mapping	Mapbox GL JS / Leaflet
Real-time	Socket.io
Deployment	Docker + Render / Railway

10 Expo Presentation Plan

1. **Introduction (30 sec):** Describe urban issues and their impact.
2. **Live Demo (3-4 min):**
 3. User submits report → ML prediction appears
 4. Map updates in real-time
 5. Admin dashboard shows assignments and heatmaps
6. **Results (1-2 min):** Show ML confidence, processing time, analytics.
7. **Q&A (1 min):** Explain ML model, workflow, and future improvements.

Next Steps

Create a **starter repository skeleton** with: - React frontend pages (report form, map, status view) - AdonisJS backend endpoints (dummy APIs) - FastAPI ML service placeholder - Worker + Redis queue integration - Docker-compose ready for rapid prototype

This setup allows fast development, integration of ML models, and a visually appealing, functional prototype for expo demonstration.